ARCH TECHNOLOGIES

Python Programming Internship

Month 1 Report

Task 1: Dice Rolling Game | Task 2: To-Do List Application

Full Name	Muhammad Shanzail
Internship Domain	Python Programming
Email Address	shanzailarif012@gmail.com
Phone Number	03183779968
Submission	Month 1
GitHub Repository	github.com/shanzailarif012-jpg/Arch-Technologies-Python-Internship

October 2026

1. Introduction

This report documents the work completed for Month 1 of the Python Programming Internship at Arch Technologies. Two tasks were assigned: a Dice Rolling Game and a To-Do List Application. Both are command-line programs written in Python.

For every task, the report presents the objective, program design, pseudocode, source code, code explanation, sample output, test cases and screenshots of the working program.

1.1 Tools and Technologies

- **Python 3:** programming language used for both tasks
- **VS Code:** code editor and integrated terminal
- **Git and GitHub:** version control and hosting of the project

2. Task 1: Dice Rolling Game

2.1 Objective

Write a program that simulates rolling a pair of dice. Each time the program rolls, it randomly generates two numbers between 1 and 6 (inclusive), displays the results and asks the user whether they would like to roll again.

2.2 Program Design

The program is split into small functions so that every part has one clear job.

Function	Purpose
roll_dice()	Generates two random numbers from 1 to 6 and returns them
show_result(die1, die2)	Prints both dice and their total
ask_roll_again()	Asks the user to roll again and validates the answer
main()	Controls the game loop and the welcome and goodbye messages

2.3 Pseudocode

```
IMPORT random module

FUNCTION roll_dice():
    die1 = random integer from 1 to 6
    die2 = random integer from 1 to 6
    RETURN die1, die2

FUNCTION show_result(die1, die2):
    PRINT die1, die2 and their total

FUNCTION ask_roll_again():
    LOOP forever:
        answer = ask user "Roll again? (yes/no)"
        clean answer (remove spaces, make lowercase)
        IF answer is "yes" or "y": RETURN True
        ELSE IF answer is "no" or "n": RETURN False
        ELSE: PRINT "Invalid input"
```

```
FUNCTION main():
    PRINT welcome message
    LOOP forever:
        die1, die2 = roll_dice()
        show_result(die1, die2)
        IF ask_roll_again() is False: BREAK
    PRINT goodbye message

IF file is run directly: CALL main()
```

2.4 Source Code

File: `main.py`

```
# Task 1 Dice-Rolling-Game

import random

def roll_dice():
    die1 = random.randint(1, 6)
    die2 = random.randint(1, 6)
    return die1, die2


def show_result(die1 , die2):
    print(f"Die 1: {die1} , Die 2: {die2}")
    # print("Die 1: " , die1)
    # print("Die 2: ", die2)
    print("Total: " , die1 + die2)


def ask_roll_again():
    while True:
        answer = input("Roll again? (yes/no): ").strip().lower()
        if answer == "yes" or answer == "y":
            return True
        elif answer == "no" or answer == "n":
            return False
        else:
            print("Invalid Input, Please Type yes or no")



def main():
    print("Welcome Dice-Rolling-Game")
    while True:
        die1 , die2 = roll_dice()
        show_result(die1, die2)
        if ask_roll_again() is False:
            break
    print("Goodbye")

if __name__ == "__main__":
    main()
```

2.5 Code Explanation

- **random.randint(1, 6):** returns a whole number from 1 to 6, both ends included, so each die is always a valid face.
- **roll_dice():** returns both values together as a pair, which main() unpacks into die1 and die2.
- **show_result():** uses an f-string to print the two dice on one line, then prints the total.
- **ask_roll_again():** uses strip() and lower() so inputs like "YES" or " n " work. A while True loop keeps asking until the answer is yes/y (returns True) or no/n (returns False), so invalid input never crashes the program.
- **main():** prints a welcome message, then loops: roll, show the result, ask to roll again, and break when the user says no.
- **if __name__ == "__main__":** makes the game start only when the file is run directly, not when it is imported.

2.6 Sample Output

```
Welcome Dice-Rolling-Game
Die 1: 2 , Die 2: 6
Total:  8
Roll again? (yes/no): yes
Die 1: 6 , Die 2: 4
Total:  10
Roll again? (yes/no): abc
Invalid Input, Please Type yes or no
Roll again? (yes/no): n
Goodbye
```

2.7 Test Cases

#	Test	Input	Expected Result
1	Normal roll	Run the program	Two numbers from 1 to 6 and their total are shown
2	Roll again	yes, y, YES	A new roll is performed
3	Stop the game	no, n, " n "	Goodbye message is shown and the program ends
4	Invalid answer	abc	"Invalid Input" message, then asks again
5	Empty answer	Press Enter	"Invalid Input" message, then asks again
6	Range check	Roll many times	Each die is always between 1 and 6

2.8 Screenshots



Figure 1: Dice Rolling Game: terminal output with re-rolls and exit

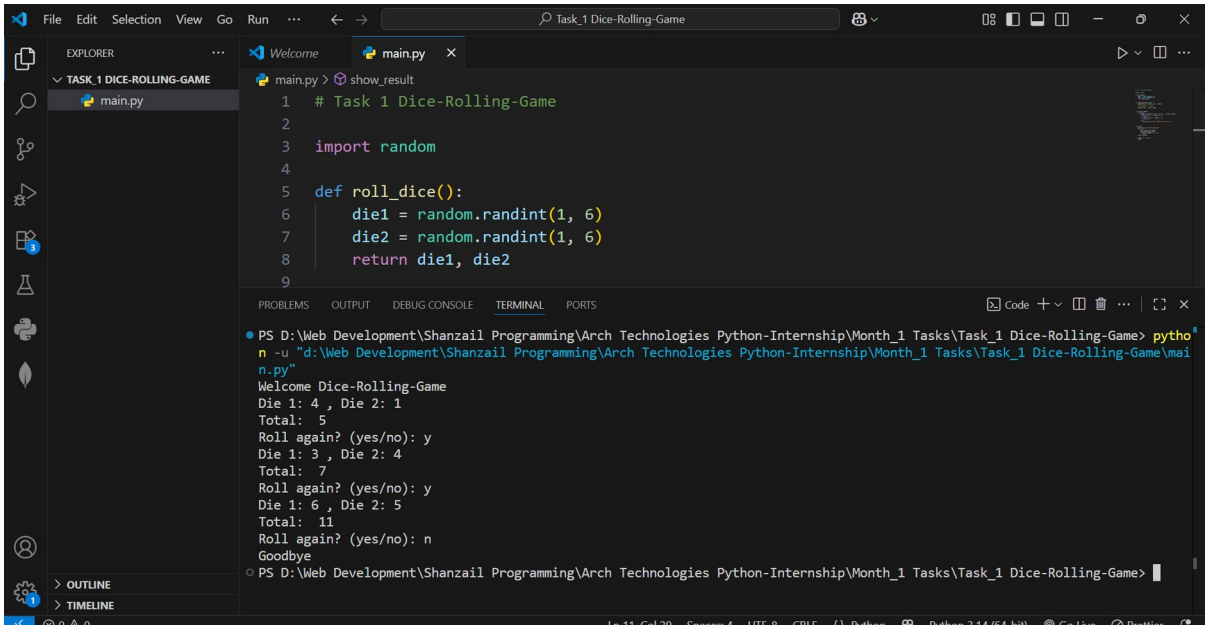


Figure 2: Dice Rolling Game: source code and output in VS Code

3. Task 2: To-Do List Application

3.1 Objective

Write a program that creates a simple To-Do List application. The user can add tasks, view the current list of tasks and remove tasks once they are completed. The program also displays a progress bar.

3.2 Program Design

Tasks are removed from the list when they are completed, so the list alone cannot show progress. The program therefore keeps two counters: total_added (all tasks ever added) and completed (tasks removed as done). Progress is calculated as completed / total_added x 100.

Function	Purpose
add_task()	Adds a new task after checking that it is not empty
view_tasks()	Prints the numbered list of tasks
remove_task()	Removes a task by number and counts it as completed
show_progress()	Calculates the percentage and prints a 20-character progress bar
show_menu()	Prints the menu options
main()	Controls the menu loop

3.3 Pseudocode

```
tasks = empty list
total_added = 0
completed = 0

FUNCTION add_task():
    text = ask user for task, remove extra spaces
    IF text is empty: PRINT error and RETURN
    append text to tasks; total_added = total_added + 1

FUNCTION view_tasks():
    IF tasks is empty: PRINT "No tasks yet" and RETURN
    FOR each task with number starting at 1: PRINT number and task

FUNCTION remove_task():
    IF tasks is empty: PRINT "Nothing to remove" and RETURN
    view_tasks()
    choice = ask user for task number
    IF choice is not a valid number OR out of range: PRINT error and RETURN
    remove that task from tasks; completed = completed + 1

FUNCTION show_progress():
    IF total_added is 0: percent = 0
    ELSE: percent = completed / total_added * 100
    filled = percent scaled to bar length (20)
    PRINT bar of filled blocks + empty blocks, and the percentage

FUNCTION main():
    LOOP forever:
        show_menu(); choice = ask user
```

```
        IF 1: add_task()  ELSE IF 2: view_tasks()  ELSE IF 3: remove_task()
        ELSE IF 4: BREAK  ELSE: PRINT "Invalid option"
        show_progress()
    PRINT goodbye message
```

3.4 Source Code

File: `main.py`

```
# Task 2 To-Do-List Application

tasks = []
total_added = 0
completed = 0

def add_task():
    global total_added
    text =input("Enter task: ").strip()
    if text == "":
        print("Task Cannot be empty")
        return
    tasks.append(text)
    total_added = total_added + 1
    print("Task Added")


def view_tasks():
    if tasks == []:
        print("No Task yet")
        return

    for number, task in enumerate(tasks, start=1):
        print(number, task)


def remove_task():
    global completed
    if tasks == []:
        print("Nothing to remove")
        return
    view_tasks()
    try:
        choice = int(input("Enter task number: "))
    except ValueError:
        print("Invalid number")
        return

    if choice < 1 or choice > len(tasks):
        print("Invalid number")
        return

    tasks.pop(choice - 1)
    completed = completed + 1
    print("Task Completed and Removed")


def show_progress():
```

```
    if total_added == 0:
        percent = 0
    else:
        percent = completed / total_added * 100
    filled = int(percent / 100 * 20)
    empty = 20 - filled
    print("" * filled + "" * empty)
    print(f"Progress: {percent:.1f}%")


def show_menu():
    print("Options: 1. Add\n2. View\n3. Remove\n4. Exit")


def main():
    while True:
        show_menu()
        try:
            choice = int(input("Enter Task Choice: "))
        except ValueError:
            print("Invalid Option! Please enter a number.")
            continue
        if choice == 1: add_task()
        elif choice == 2: view_tasks()
        elif choice == 3: remove_task()
        elif choice == 4: break
        else:
            print("Invalid Option")
        show_progress()
    print("GoodBye TakeCare")


if __name__ == "__main__":
    main()
```

3.5 Code Explanation

- **tasks, total_added, completed:** a list and two counters that hold the program state while it runs.
- **add_task():** reads the task, strips extra spaces, rejects empty text, then appends it and increases total_added. The global keyword lets the function change the counter.
- **view_tasks():** prints "No Task yet" for an empty list, otherwise numbers the tasks from 1 using enumerate(tasks, start=1).
- **remove_task():** shows the list, reads a task number inside try/except ValueError, checks it is within range, removes it with pop(choice - 1) and increases completed.
- **show_progress():** returns 0% when nothing has been added (avoids division by zero). Otherwise it converts the percentage into a number of filled blocks out of 20 and prints filled and empty blocks with the percentage.
- **main():** shows the menu in a loop, validates that the choice is a number, calls the matching function and shows the progress bar after every action. Option 4 exits the loop.

3.6 Sample Output

```
Options: 1. Add
2. View
3. Remove
4. Exit
Enter Task Choice: 1
Enter task: Buy milk
Task Added
░░░░░░░░░░░░░░░░░░░░░░░░
Progress: 0.0%
... (menu is shown again before every choice)
Enter Task Choice: 1
Enter task: Study Python
Task Added
░░░░░░░░░░░░░░░░░░░░░░░░
Progress: 0.0%
...
Enter Task Choice: 2
1 Buy milk
2 Study Python
░░░░░░░░░░░░░░░░░░░░░░░░
Progress: 0.0%
...
Enter Task Choice: 3
1 Buy milk
2 Study Python
Enter task number: 1
Task Completed and Removed
████████░░░░░░░░░░░░░░░░
Progress: 50.0%
...
Enter Task Choice: 4
GoodBye TakeCare
```

3.7 Test Cases

#	Test	Input	Expected Result
1	Add a task	1, then Buy milk	"Task Added"; progress bar updates
2	Add an empty task	1, then press Enter	"Task Cannot be empty"
3	View tasks	2	Numbered list of tasks
4	View an empty list	2 with no tasks	"No Task yet"
5	Remove a task	3, then 1	Task removed; with 2 added and 1 done the bar shows 50.0%
6	Invalid task number	abc, 0, 99	"Invalid number"
7	Remove from an empty list	3 with no tasks	"Nothing to remove"
8	Invalid menu option	9	"Invalid Option"
9	Non-numeric menu input	abc	"Invalid Option! Please enter a number."

#	Test	Input	Expected Result
10	Exit	4	Goodbye message and the program ends

3.8 Limitation

Tasks are stored in memory only, so the list and the progress reset when the program closes. Saving tasks to a file could be added as a future improvement.

3.9 Screenshots

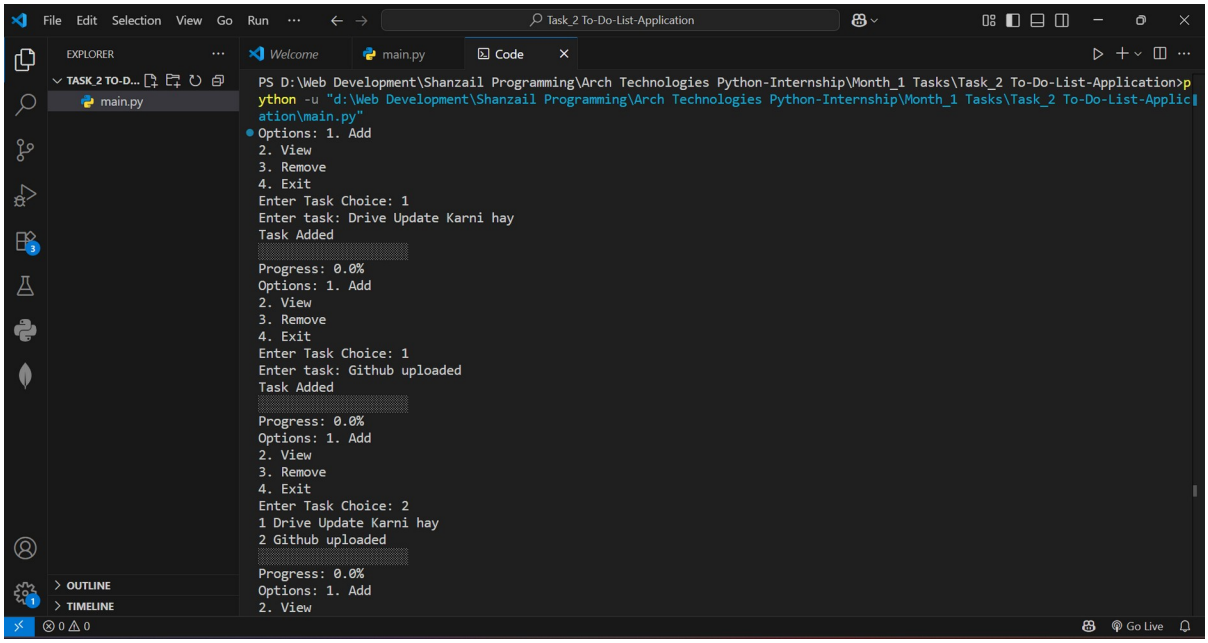


Figure 3: To-Do List: adding and viewing tasks

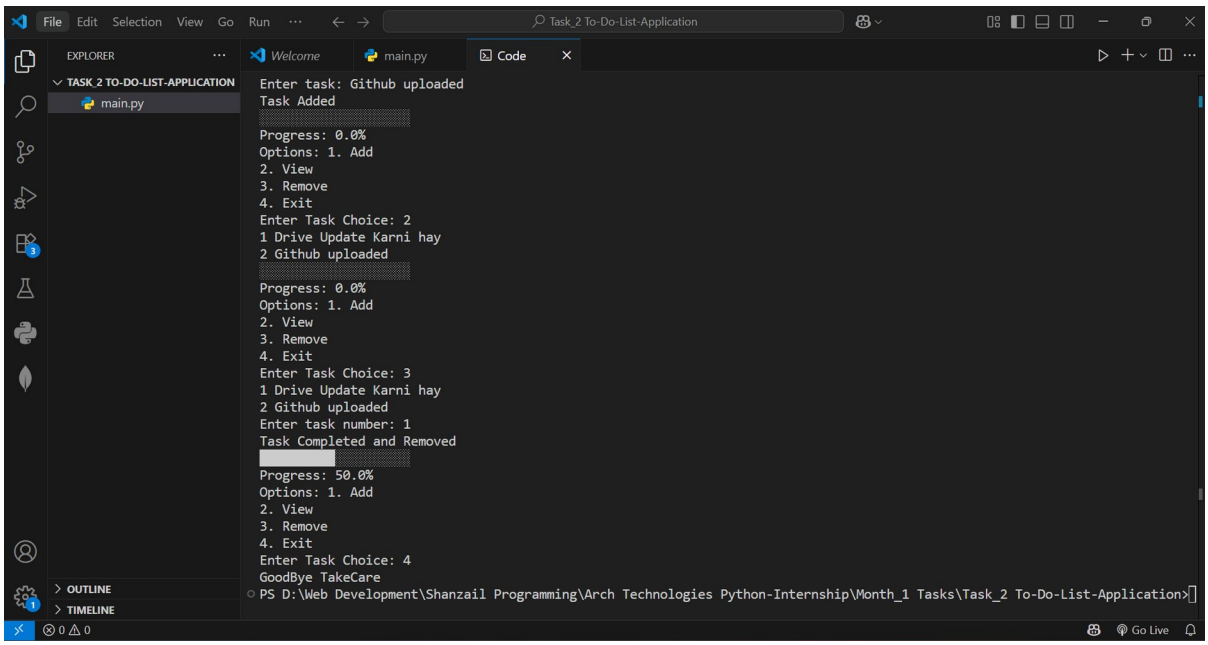


Figure 4: To-Do List: removing a task, progress bar at 50% and exit

4. GitHub Repository

Both tasks are uploaded to a public GitHub repository, with one folder per task containing the code, a README and screenshots.

Repository: https://github.com/shanzailarif012-jpg/Arch-Technologies-Python-Internship

4.1 Repository Structure

```
Arch-Technologies-Python-Internship/
├── README.md
└── Month_1 Tasks/
    ├── Task_1 Dice-Rolling-Game/
    │   ├── main.py
    │   ├── READme.md
    │   └── Screenshots/
    └── Task_2 To-Do-List-Application/
        ├── main.py
        ├── READme.md
        └── Screenshots/
```

4.2 Git Workflow Used

Command	Purpose
git init	Creates a local repository
git add .	Stages the files
git commit -m "..."	Saves the changes with a message
git branch -M main	Renames the branch to main
git remote add origin <url>	Connects the local repository to GitHub
git push -u origin main	Uploads the commits to GitHub

5. Challenges and Learnings

- **Tracking progress after removal:** completed tasks leave the list, so two counters (total_added and completed) were used to calculate progress correctly.
- **Input validation:** try/except ValueError, range checks and loops that ask again keep both programs from crashing on wrong input.
- **Code structure:** splitting the programs into small functions made the code easier to read, test and explain.
- **Git and GitHub:** when a push was rejected because GitHub had a newer commit, it was fixed with git pull --rebase origin main followed by git push.

6. Conclusion

Both Month 1 tasks were completed successfully. The Dice Rolling Game and the To-Do List Application run as required, handle invalid input safely, and are organised into clear functions. The complete work is available on GitHub for review.